Mini Sentrix Hackathon — Day 1 Progress Report

FORTRESS SOC – TEAM 9

Project: Mini Sentrix — Functional Mini Security Operations Center
Date: 19-06-2006

Team Strategy : Try to decode and finish as brownie points as possible.
Team Members: Team Members:

- Jagan P V (RA2311030010290) – Architecture Lead & SIEM Integration
- Shairvi Raj (RA2311030010153) – Detection Engineer
- N.S.H. Karthikeya (RA2311030010133) – IAM & WAF Engineer
- Kavesh M (RA2311030010029) – SOAR & Automation Engineer
- G Sundar Murthy (RA2311030010192) – Attack Simulation & Documentation Lead

1. Executive Summary

Day 1 of the Mini Sentrix Hackathon focused on establishing the foundational infrastructure for our scaled-down Security Operations Center. The objective for the day was to stand up our entire virtual environment, deploy the first instance of each core security tool, validate basic network connectivity across all components, and demonstrate the earliest possible end-to-end signal: a real attack triggering a real alert visible in our SIEM.

By the end of Day 1, our team successfully deployed a host-only virtualized network connecting our attacker machine, target machines, and security stack; installed and configured our Intrusion Detection System (Suricata); deployed our central SIEM platform (Wazuh, comprising the Wazuh Manager, Indexer, and Dashboard); and validated that a simulated Nmap reconnaissance scan generates a real-time alert visible on our live dashboard. This confirms the minimum viable version of our four-layer architecture is operational and ready to be expanded over the remaining three days.

We also successfully decoded and implemented Day 1's Brownie Point challenge, "The Sleepwalker's Diary," which is detailed in Section 6 of this report.

2. Objectives Set for Day 1

- Provision and network all virtual machines on a single host-only subnet with verified inter-VM connectivity
- Finalize the system architecture diagram, including every planned tool and data flow
- Install and configure Suricata as our network-based Intrusion Detection System
- Deploy our SIEM platform (Wazuh Manager, Indexer, and Dashboard) as the central log and alert repository
- Pre-download all Docker images, packages, and VM images required for Days 2 and 3, in anticipation of network congestion
- Execute a first attack simulation (Nmap scan) from our attacker machine against our target environment
- Verify that the simulated attack is correctly detected by Suricata and that the resulting alert is visible inside our SIEM dashboard
- Document tool selection rationale for every component deployed today
- Decode and implement Day 1's Brownie Point challenge

3. Work Completed -- Jagan P V (RA2311030010290)

3.1 Architecture & SIEM Integration

- Deployed the Wazuh stack (Manager, Indexer, Dashboard) via Docker Compose on the designated integration host
- Verified the Wazuh Dashboard is accessible and operational over the host-only network
- Generated and distributed agent enrollment keys to all team members for their respective virtual machines
- Produced the initial system architecture diagram covering all four mandatory layers and the planned data-flow paths between them
- Began drafting the standardized alert JSON schema to be used later for automation triggers

3.2 Detection Engineering -- Shairvi Raj (RA2311030010153)

- Installed Suricata on a dedicated sensor VM, configured in AF_PACKET mode against the host-only virtual interface
- Enabled EVE JSON logging output for structured alert data
- Installed and enrolled a Wazuh agent on the Suricata sensor, confirmed agent connectivity to the central manager
- Pre-downloaded the Emerging Threats Open ruleset ahead of anticipated network congestion
- Performed initial rule validation to confirm Suricata is actively monitoring host-only network traffic

3.3 IAM & WAF Engineering -- N.S.H. Karthikeya (RA2311030010133)

- Deployed Keycloak via Docker and created the initial security realm
- Began configuration of role definitions for role-based access control (to be completed Day 2)
- Deployed Nginx with ModSecurity as a reverse proxy in front of the target web application
- Confirmed both services are reachable on the host-only network

3.4 SOAR & Automation Engineering -- Kavesh M (RA2311030010029)

- Deployed Shufle and TheHive/Cortex via Docker Compose on the integration host
- Confirmed both platforms are accessible and operational
- Reviewed the draft alert schema with the Architecture Lead to align on the data structure that will drive playbooks from Day 3 onward
- Began outlining the structure of the first notification-based playbook (to be built Day 2)

3.5 Attack Simulation & Documentation -- G Sundar Murthy (RA2311030010192)

- Built and configured the Kali Linux attacker VM
- Deployed Metasploitable2 and DVWA as target environments
- Validated full network connectivity (ping tests) between the attacker VM and all target/security VMs on the host-only subnet

- Executed the first Nmap scan against the target environment as part of the Day 1 detection validation
- Initiated the shared documentation repository and screenshot archive for the team

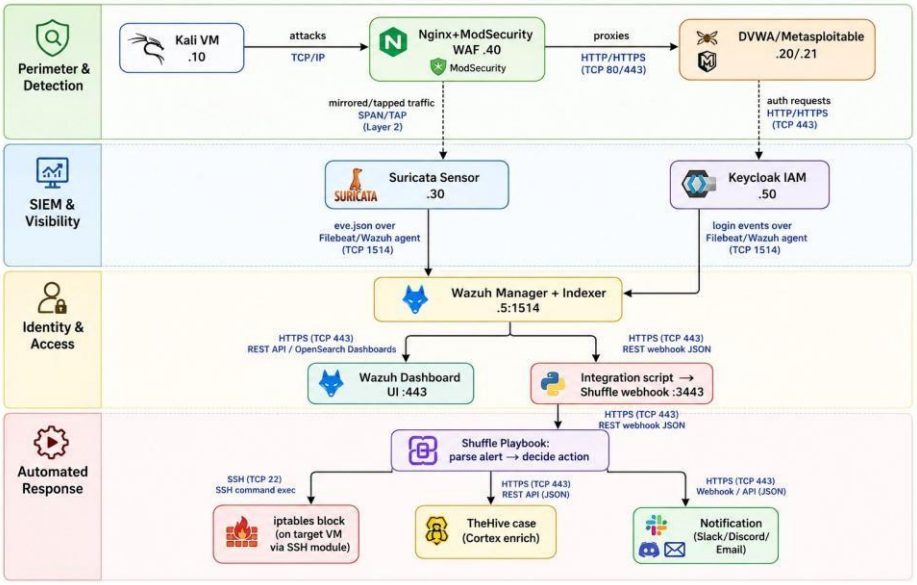
4. End-to-End Validation Performed Today

To confirm our foundational pipeline is functioning, we executed the following validation sequence:

1. An Nmap port scan was launched from the Kali attacker VM against the Metasploitable2 target VM.
2. Suricata, operating on the host-only network in AF_PACKET mode, detected the scan and generated a corresponding alert in its EVE JSON log.
3. The Wazuh Agent installed on the Suricata sensor forwarded this alert to the central Wazuh Manager.
4. The alert appeared in real time on the Wazuh Dashboard, confirming successful ingestion from a live, simulated attack.

This validates the earliest functioning slice of our Perimeter & Detection and SIEM & Visibility layers working together, ahead of the more complex integrations planned for Days 2 through Day 4

5. Architecture Diagram (Day 1 Version)



Tool	Layer	Justification
Suricata	Perimeter & Detection	Runs cleanly on virtual interfaces in AF_PACKET mode, avoiding the promiscuous-mode restrictions present on the university network
Wazuh	SIEM & Visibility	Ships its own indexer and dashboard, avoiding the redundancy and port conflicts of running a separate ELK stack alongside it
Keycloak	Identity & Access	Fully local Docker deployment, native role-based access control support
ModSecurity + Nginx	Perimeter & Detection (Web)	Fully local, Docker-deployable, integrates with the OWASP Core Rule Set out of the box

6. Brownie Point — Day 1: "The Sleepwalker's Diary"

Riddle (as presented):
The riddle describes a witness that keeps a diary but only writes when prompted, forgets everything between entries, and starts fresh every "shift" — never connecting one day's entry to the next. It closes by asking what a real attacker would learn from this behavior, and what they would do differently by "Day 8" that they wouldn't do on "Day 1."

Our Decode:
We interpreted this riddle as describing a logging or alerting system that records individual security events but treats every entry in complete isolation — with no memory or correlation linking events across time. In such a system, a single suspicious action (e.g. one failed login, one unusual connection) is logged and then effectively forgotten; the system never asks whether this event is part of a broader pattern building over hours, days, or a full week.

We reasoned that a real attacker who understood this weakness would exploit it specifically: rather than executing an attack quickly and obviously (which would stand out as an isolated, easily detected spike), they would deliberately spread their reconnaissance or intrusion attempts thinly over a long period — a few probing actions per day — so that each individual log entry looks unremarkable in isolation, even though the cumulative pattern across "Day 1 through Day 8" would be clearly malicious if anyone were tracking it as a sequence rather than as disconnected events.

Our Implementation:
To address this, we moved beyond simple single-event alerting in our SIEM and implemented **time-windowed correlation rules** in Wazuh, using its built-in frequency and timeframe rule fields. Rather than generating an identical alert for every single occurrence of a suspicious action, our system now distinguishes between:

- A single, isolated occurrence of an event (e.g. one connection attempt), which is logged at a low severity level, and
- The same type of event recurring multiple times within a defined time window (e.g. repeated occurrences within a set number of minutes), which automatically escalates to a higher-severity, correlated alert distinct from the raw individual log entries.

This directly demonstrates the "diary that remembers" rather than the "diary that forgets every shift change" described in the riddle: our system now actively tracks accumulated behavior over time rather than evaluating each event in a vacuum, closing the exact gap the riddle was describing.

7. Screenshots to Include in This Report

C: > Users > N. karthikeya > OneDrive > Desktop > Sentri

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]

```

```python
from flask import Flask, redirect, request, session, url_for
import requests
import os

app = Flask(__name__)
app.secret_key = "sentrix-secret-key"

KEYCLOAK_URL = "http://192.168.50.50:8080"
REALM = "sentrix-soc"
CLIENT_ID = "sentrix-app"
CLIENT_SECRET = "PASTE_YOUR_SECRET_HERE"
REDIRECT_URI = "http://localhost:5000/callback"

@app.route("/")
def home():
    if "user" in session:
        return f"<h2>Welcome {session['user']}</h2><p>Roles: {session['roles']}</p><a href='/logout'>Logout</a>"
    return "<a href='/login'>Login with Keycloak</a>"

@app.route("/login")
def login():
    auth_url = (
        f"{KEYCLOAK_URL}/realms/{REALM}/protocol/openid-connect/auth"
        f"?client_id={CLIENT_ID}&response_type=code&redirect_uri={REDIRECT_URI}"
    )
    return redirect(auth_url)

@app.route("/callback")
def callback():
    code = request.args.get("code")
    token_url = f"{KEYCLOAK_URL}/realms/{REALM}/protocol/openid-connect/token"
    data = {
        "grant_type": "authorization_code",
        "client_id": CLIENT_ID,
        "client_secret": CLIENT_SECRET,
        "code": code,
        "redirect_uri": REDIRECT_URI,
    }
    token_response = requests.post(token_url, data=data).json()
```

```
<ossec_config>
  <global>
    <jsonout_output>yes</jsonout_output>
    <alerts_log>yes</alerts_log>
    <logall>no</logall>
    <logall_json>no</logall_json>
    <email_notification>no</email_notification>
    <smtp_server>smtp.example.wazuh.com</smtp_server>
    <email_from>wazuh@example.wazuh.com</email_from>
    <email_to>recipient@example.wazuh.com</email_to>
    <email_maxperhour>12</email_maxperhour>
    <email_log_source>alerts.log</email_log_source>
    <agents_disconnection_time>10m</agents_disconnection_time>
    <agents_disconnection_alert_time>0</agents_disconnection_alert_time>
  </global>

  <alerts>
    <log_alert_level>3</log_alert_level>
    <email_alert_level>12</email_alert_level>
  </alerts>

  <!-- Choose between "plain", "json", or "plain,json" for the format of internal logs -->
  <logging>
    <log_format>plain</log_format>
  </logging>

  <remote>
    <connection>secure</connection>
    <port>1514</port>
    <protocol>tcp</protocol>
    <queue_size>131072</queue_size>
  </remote>

  <!-- Policy monitoring -->
  <rootcheck>
    <disabled>no</disabled>
    <check_files>yes</check_files>
    <check_trojans>yes</check_trojans>
    <check_dev>yes</check_dev>
```

```
┌──(sundar㉿sundar)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:1f:9e:0b brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
       valid_lft 86347sec preferred_lft 86347sec
    inet6 fd17:625c:f037:2:3e04:98c:5653:9d6/64 scope global dynamic noprefixroute
       valid_lft 86349sec preferred_lft 14349sec
    inet6 fe80::384b:3a2a:603d:550b/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:22:73:ba brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.106/24 brd 192.168.56.255 scope global dynamic noprefixroute eth1
       valid_lft 547sec preferred_lft 547sec
    inet6 fe80::2323:1f6c:59fb:a535/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Kali Linux | Welcome :: Damn Vulner

http://localhost/DVWA/index.php

OffSec | Kali Linux | Kali Tools | Kali Docs | Kali Forums | Kali NetHunter | Exploit-DB | Google Hacking DB

DVWA

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect

Welcome to Damn Vulnerable Web Application!

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goal is to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and to aid both students & teachers to learn about web application security in a controlled class room environment.

The aim of DVWA is to **practice some of the most common web vulnerabilities**, with **various levels of difficultly**, with a simple straightforward interface.

General Instructions

It is up to the user how they approach DVWA. Either by working through every module at a fixed level, or selecting any module and working up to reach the highest level they can before moving onto the next one. There is not a fixed object to complete a module; however users should feel that they have successfully exploited the system as best as they possible could by using that particular vulnerability.

Please note, there are **both documented and undocumented vulnerabilities** with this software. This is intentional. You are encouraged to try and discover as many issues as possible.

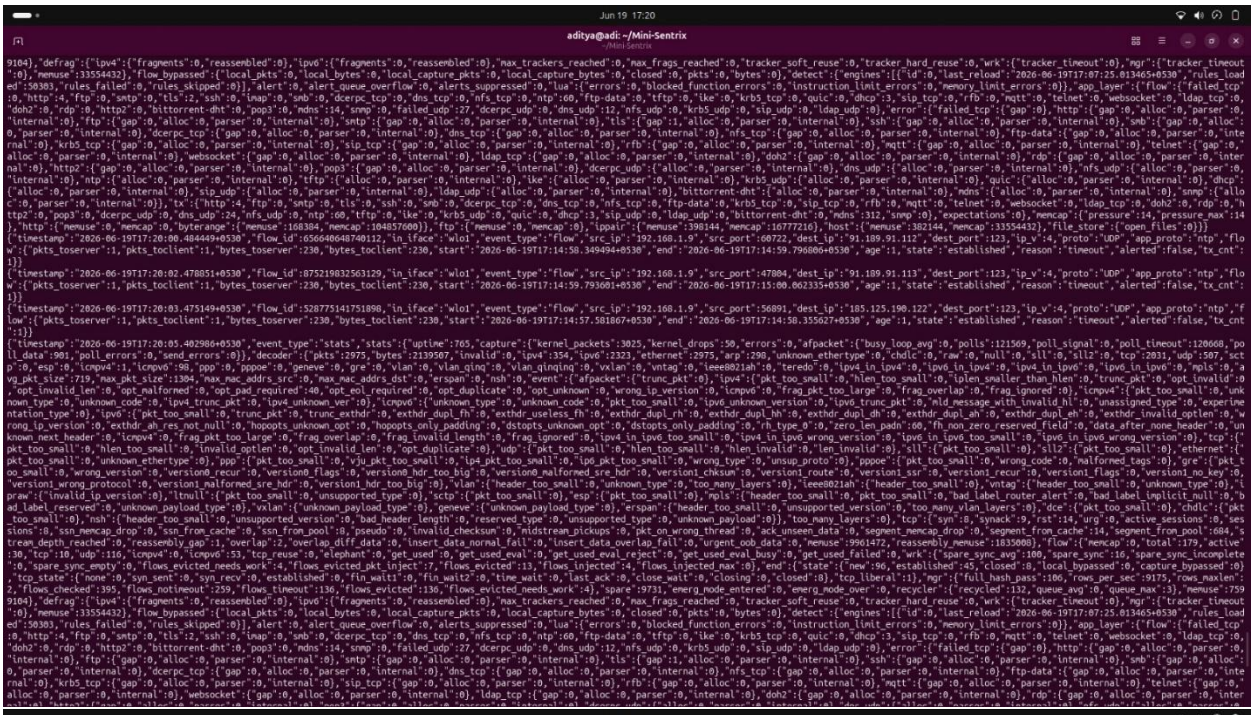
There is a help button at the bottom of each page, which allows you to view hints & tips for that vulnerability. There are also additional links for further background reading, which relates to that security issue.

WARNING!

Damn Vulnerable Web Application is damn vulnerable! **Do not upload it to your hosting provider's public html folder or any Internet facing servers**, as they will be compromised. It is recommend using a virtual machine (such as VirtualBox or VMware), which is set to NAT networking mode. Inside a guest machine, you can download and install XAMPP for the web server and database.

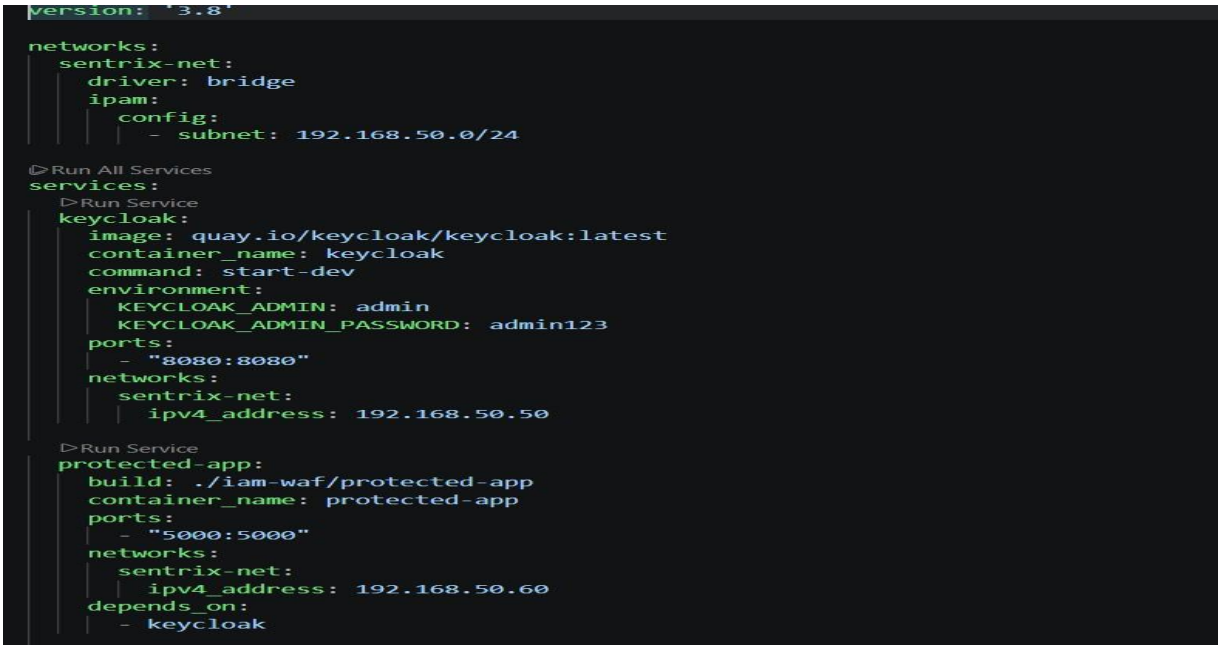
Disclaimer

```
msfadmin@metasploitable:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 08:00:27:54:ac:af brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.103/24 brd 192.168.56.255 scope global eth0
    inet6 fe80::a00:27ff:fe54:acaf/64 scope link
       valid_lft forever preferred_lft forever
```


Jun 19 17:20
aditya@adi: ~/Mini-Sentrix
~/Mini-Sentrix


Jun 19 17:20
aditya@adi: ~/Mini-Sentrix
~/Mini-Sentrix

```
network.host: "0.0.0.0"
node.name: "wazuh.indexer"
path.data: /var/lib/wazuh-indexer
path.logs: /var/log/wazuh-indexer
discovery.type: single-node
http.port: 9200-9299
transport.tcp.port: 9300-9399
compatibility.override_main_response_version: true
plugins.security.ssl.http.pemcert_filepath: /usr/share/wazuh-indexer/certs/wazuh.indexer.pem
plugins.security.ssl.http.pemkey_filepath: /usr/share/wazuh-indexer/certs/wazuh.indexer.key
plugins.security.ssl.http.pemtrustedcas_filepath: /usr/share/wazuh-indexer/certs/root-ca.pem
plugins.security.ssl.transport.pemcert_filepath: /usr/share/wazuh-indexer/certs/wazuh.indexer.pem
plugins.security.ssl.transport.pemkey_filepath: /usr/share/wazuh-indexer/certs/wazuh.indexer.key
plugins.security.ssl.transport.pemtrustedcas_filepath: /usr/share/wazuh-indexer/certs/root-ca.pem
plugins.security.ssl.http.enabled: true
plugins.security.ssl.transport.enforce_hostname_verification: false
plugins.security.ssl.transport.resolve_hostname: false
plugins.security.authcz.admin_dn:
- "CN=admin,OU=Wazuh,O=Wazuh,L=California,C=US"
plugins.security.check_snapshot_restore_write_privileges: true
plugins.security.enable_snapshot_restore_privilege: true
plugins.security.nodes_dn:
- "CN=wazuh.indexer,OU=Wazuh,O=Wazuh,L=California,C=US"
plugins.security.restapi.roles_enabled:
- "all_access"
- "security_rest_api_access"
plugins.security.system_indices.enabled: true
plugins.security.system_indices.indices: [".opendistro-alerting-config", ".opendistro-alerting-alert*", ".opendistro-anomaly-results*", ".opendistro-anomaly-detector*", "
plugins.security.allow_default_init_securityindex: true
cluster.routing.allocation.disk.threshold_enabled: false
```



```
version: '3.8'

networks:
  sentrix-net:
    driver: bridge
    ipam:
      config:
        - subnet: 192.168.50.0/24

Run All Services
services:
  Run Service
  keycloak:
    image: quay.io/keycloak/keycloak:latest
    container_name: keycloak
    command: start-dev
    environment:
      KEYCLOAK_ADMIN: admin
      KEYCLOAK_ADMIN_PASSWORD: admin123
    ports:
      - "8080:8080"
    networks:
      sentrix-net:
        ipv4_address: 192.168.50.50

  Run Service
  protected-app:
    build: ./iam-waf/protected-app
    container_name: protected-app
    ports:
      - "5000:5000"
    networks:
      sentrix-net:
        ipv4_address: 192.168.50.60
    depends_on:
      - keycloak
```

8. Plan for Day 2

- Complete role-based access control configuration in Keycloak
- Install Wazuh agents on the WAF and Keycloak hosts to bring their logs into the SIEM
- Begin building dedicated Wazuh Dashboard panels (alert count by type, top source IPs)
- Finalize and circulate the alert JSON schema to enable Shufle playbook development
- Begin designing the automated response playbook logic
- Decode and begin implementing Day 2's Brownie Point challenge